

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมและ GRASP Patterns

ระบบ Game Catalog ใช้สถาปัตยกรรมแบบ Layered Architecture โดยแบ่งการทำงานออกเป็น Entity, Repository, Service และ Controller เพื่อให้แต่ละส่วนมีหน้าที่ชัดเจน

- Entity (Game) ทำหน้าที่เก็บข้อมูลของเกมและเชื่อมโยงกับตาราง games ใน PostgreSQL
- Repository (GameRepository) ทำหน้าที่ติดต่อและจัดการข้อมูลในฐานข้อมูล โดยใช้ Spring Data JPA
- Service (GameService) ทำหน้าที่จัดการ Business Logic เช่น การเพิ่ม แก้ไข ลบ และ ค้นหาข้อมูลเกม รวมถึงการคำนวณราคาหลังส่วนลด

Controller (GameController) ทำหน้าที่รับ HTTP Request จากผู้ใช้ เรียกใช้งาน Service และส่งข้อมูลไปแสดงผลผ่าน Thymeleaf

1.2 High-Level SOLID Principles

- Single Responsibility Principle (SRP) แต่ละคลาสมีหน้าที่เฉพาะ เช่น Controller จัดการ HTTP Request, Service จัดการ Business Logic, Repository จัดการฐานข้อมูล และ Strategy จัดการการคำนวณส่วนลด
- Open/Closed Principle (OCP) ระบบสามารถเพิ่ม Strategy สำหรับส่วนลดรูปแบบใหม่ได้โดยไม่ต้องแก้ไขโครงสร้างหลักของระบบ
- Liskov Substitution Principle (LSP) Strategy แต่ละประเภทสามารถใช้แทน DiscountStrategy ได้ เนื่องจากมีการทำงานตาม Interface เดียวกัน
- Interface Segregation Principle (ISP) DiscountStrategy มีเฉพาะเมธอดที่จำเป็นสำหรับการคำนวณส่วนลด ทำให้คลาสที่นำไปใช้งานไม่ต้องพึ่งพาเมธอดที่ไม่จำเป็น
- Dependency Inversion Principle (DIP) ระบบใช้ Dependency Injection โดยให้ Spring จัดการและส่ง Dependency ผ่าน Constructor แทนการสร้าง Object ภายในคลาสโดยตรง

1.3 Strategy Pattern

ระบบใช้ Strategy Pattern สำหรับจัดการการคำนวณส่วนลดราคาของเกม โดยแยกวิธีคำนวณส่วนลดแต่ละประเภทออกเป็นคลาสที่แตกต่างกัน ทำให้สามารถเลือกวิธีคำนวณได้ตาม discountType

- DiscountStrategy เป็น Interface ที่กำหนดรูปแบบการคำนวณราคา
- NoDiscountStrategy ใช้สำหรับราคาปกติ ไม่มีส่วนลด
- StudentDiscountStrategy ใช้สำหรับส่วนลดนักศึกษา 10%
- SeasonalSaleStrategy ใช้สำหรับส่วนลดเทศกาล 20%
- DiscountContext ทำหน้าที่เลือก Strategy ที่เหมาะสมตามประเภทส่วนลด แล้วเรียกใช้เพื่อคำนวณราคาราคาสูทธิ

การออกแบบนี้ช่วยให้แต่ละ Strategy มีหน้าที่เฉพาะของตัวเอง และสามารถเพิ่ม Strategy ใหม่ได้โดยสร้างคลาสที่ implements DiscountStrategy โดยไม่จำเป็นต้องแก้ไขโค้ดส่วนอื่นที่มีอยู่เดิม ซึ่งสอดคล้องกับ Open/Closed Principle

1.4 Layered Architecture

ระบบถูกแบ่งออกเป็นหลาย Layer เพื่อแยกหน้าที่ของแต่ละส่วนอย่างชัดเจน โดยแต่ละ Layer รับผิดชอบงานเฉพาะของตัวเอง

- Controller Layer รับผิดชอบจัดการ HTTP Request จากผู้ใช้ เช่น การเพิ่ม แก้ไข แสดง และลบข้อมูลเกม แล้วส่งต่อการทำงานให้ Service
- Service Layer รับผิดชอบ Business Logic ของระบบ เช่น การจัดการข้อมูลเกมและการคำนวณราคาหลังหักส่วนลดผ่าน Strategy Pattern
- Repository Layer รับผิดชอบติดต่อฐานข้อมูล PostgreSQL ผ่าน Spring Data JPA เช่น การเพิ่ม ค้นหา แก้ไข และลบข้อมูล
- Entity (Game) เป็นตัวแทนข้อมูลเกมและใช้สำหรับ Mapping กับตาราง games ในฐานข้อมูล

การแยก Service ออกจาก Controller และ Repository ช่วยให้แต่ละส่วนมี High Cohesion เพราะแต่ละคลาสมีหน้าที่ที่เกี่ยวข้องกันโดยตรง และมี Low Coupling เพราะแต่ละ Layer ไม่ต้องจัดการรายละเอียดของ Layer อื่นโดยตรง ทำให้ระบบแก้ไขและบำรุงรักษาได้ง่ายขึ้น

1.5 Execution Flow

เมื่อผู้ใช้ส่ง HTTP Request จาก Browser ระบบจะทำงานตามลำดับ

1. Browser ส่ง HTTP Request ไปยัง GameController เช่น การเพิ่ม แก้ไข ดู หรือ ลบข้อมูลเกม
2. GameController รับ Request และส่งต่อการทำงานให้ GameService โดยใช้ Constructor Injection
3. GameService จัดการ Business Logic และเรียก GameRepository เพื่อบันทึกหรือดึงข้อมูลเกมจาก PostgreSQL
4. GameRepository ใช้ Spring Data JPA ติดต่อกับ PostgreSQL โดย Hibernate จะสร้างคำสั่ง SQL และจัดการข้อมูลในตาราง games
5. ในกรณีที่ต้องคำนวณราคาสส่วนลด GameService จะเรียก DiscountContext เพื่อเลือก Strategy ตาม discountType
6. DiscountContext เลือก Strategy ที่เหมาะสม ได้แก่ NoDiscountStrategy, StudentDiscountStrategy หรือ SeasonalSaleStrategy แล้วคำนวณราคาสุทธิ
7. ผลลัพธ์จะถูกส่งกลับจาก GameService → GameController → Thymeleaf Template
8. Thymeleaf นำข้อมูลมาแสดงผลเป็นหน้าเว็บ และส่งกลับไปยัง Browser

สรุป Flow:

Browser → Controller → Service → Repository → PostgreSQL → Service → Strategy → Controller → Thymeleaf → Browser

ส่วนที่ 2: Code Implementation & Explanation

2.1 Entity — Game.java

Game เป็น Entity สำหรับจัดเก็บข้อมูลเกมและ Map กับตาราง games ใน PostgreSQL โดยใช้ JPA Annotation

โค้ดส่วนสำคัญ:

```
package com.example.demo.model;

import jakarta.persistence.*;
import java.time.LocalDate;
import org.springframework.format.annotation.DateTimeFormat;
```

```
@Entity
@Table(name = "games")
public class Game {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String genre;
    private String platform;
    private Double rating;

    @DateTimeFormat(pattern = "yyyy-MM-dd")
    private LocalDate releaseDate;

    private Double price;
    private String discountType;

    @Transient
    private Double finalPrice;

    @Transient
    private String discountName;

    // Getters and Setters
```

@Entity ใช้กำหนดให้ Game เป็น Entity และ @Id กับ @GeneratedValue ใช้กำหนด Primary Key ที่สร้างค่าอัตโนมัติ

2.2 Repository — GameRepository.java

GameRepository ทำหน้าที่เป็น Data Access Layer สำหรับติดต่อกับฐานข้อมูล และสืบทอดความสามารถจาก JpaRepository

โค้ด:

```
package com.example.demo.repository;
```

```
import com.example.demo.model.Game;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
```

```
@Repository
```

```
public interface GameRepository extends JpaRepository<Game, Long> {
}
```

ทำให้สามารถใช้ฟังก์ชัน CRUD เช่น findAll(), findById(), save() และ deleteById() ได้โดยไม่ต้องเขียน SQL เอง

2.3 Strategy Package

ระบบใช้ Strategy Pattern สำหรับคำนวณราคาตามประเภทส่วนลด โดยมี DiscountStrategy เป็น Interface กลาง

โค้ด:

```
package com.example.demo.strategy;
```

```
public interface DiscountStrategy {
    double calculatePrice(double price);
    String getDiscountName();
}
```

มี Strategy สำหรับแต่ละรูปแบบ ได้แก่ NoDiscountStrategy, StudentDiscountStrategy และ SeasonalSaleStrategy ซึ่งแต่ละคลาสกำหนดวิธีคำนวณส่วนลดของตนเอง

DiscountContext ทำหน้าที่เลือก Strategy ตาม discountType ที่ได้รับ ทำให้สามารถเลือกวิธีคำนวณราคาได้อย่างเหมาะสม

2.4 Service — GameService.java

GameService เป็น Service Layer ที่รับผิดชอบ Business Logic และเชื่อมต่อกับ Repository และ Strategy ผ่าน Constructor Injection

โค้ดส่วน Constructor:

```
private final GameRepository gameRepository;
private final DiscountContext discountContext;
```

```

public GameService(
    GameRepository gameRepository,

    DiscountContext discountContext) {
    this.gameRepository = gameRepository;
    this.discountContext = discountContext;
}

```

ภายใน Service มีการจัดการ CRUD ได้แก่ การแสดงข้อมูลทั้งหมด การค้นหาตาม ID การเพิ่ม การแก้ไข และการลบข้อมูล รวมถึงการคำนวณราคาสุทธิผ่าน DiscountContext

2.5 Controller — GameController.java

GameController ทำหน้าที่รับ HTTP Request จากผู้ใช้และเรียกใช้งาน GameService เพื่อดำเนินการ CRUD
 โค้ดส่วน Constructor:

```

private final GameService gameService;

public GameController(GameService gameService) {
    this.gameService = gameService;
}

```

Controller กำหนด URL สำหรับการทำงานต่าง ๆ เช่น /games สำหรับแสดงรายการเกม, /games/add สำหรับเพิ่มข้อมูล, /games/edit/{id} สำหรับแก้ไข และ /games/delete/{id} สำหรับลบข้อมูล

2.6 Dependency Injection (Constructor Injection)

ระบบใช้ Constructor Injection ในการส่ง Dependency ระหว่างแต่ละ Layer โดย Spring Framework เป็นผู้สร้างและจัดการ Object ให้ ทำให้แต่ละคลาสไม่จำเป็นต้องสร้าง Dependency เอง

การเชื่อมโยงหลักของระบบคือ Controller → Service → Repository → PostgreSQL และในส่วนการคำนวณส่วนลดคือ Service → DiscountContext → DiscountStrategy

การใช้ Constructor Injection ช่วยลดการพึ่งพากันโดยตรงระหว่างคลาส ทำให้ระบบมี Low Coupling และสามารถดูแลหรือทดสอบแต่ละส่วนได้ง่ายขึ้น

ส่วนที่ 3: Web Application & Database Screenshots

หน้าจอเพิ่มเกมใหม่ (Create)

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

+

เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม

Elden Ring (673380052-5 SEC 2)

แนวเกม (Genre)

แพลตฟอร์ม (Platform)

Action RPG

PC / PS5

คะแนน (Rating)

ราคาปกติ (บาท)

% ส่วนลด (Strategy)

9.8

1790

ส่วนลดนักศึกษา (10%)

วันวางจำหน่าย (Release Date)

09/08/2026

บันทึก

ยกเลิก

หน้าจอแสดงรายการเกมทั้งหมด (Read)

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

เพิ่มเกมใหม่

เพิ่มเกมสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Elden Ring (673380052-5 SEC 2)	Action RPG	PC / PS5	★ 9.8	09/08/2026	ส่วนลดนักศึกษา (10%)	1611.00 1790.00	แก้ไข ลบ
2	Elden Ring (673380052-5 SEC 2)	Action RPG	PC / PS5	★ 9.9	10/08/2026	ส่วนลดเทศบาล (20%)	1432.00 1790.00	แก้ไข ลบ

ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

หน้าจอแก้ไขเกม (Update)

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: Elden Ring (673380052-5 SEC 2)

ชื่อเกม

Elden Ring (673380052-5 SEC 2)

แนวเกม (Genre)

Action RPG

แพลตฟอร์ม (Platform)

PC / PS5

คะแนน (Rating)

9.7

ราคาปกติ (บาท)

1790.0

% ส่วนลด (Strategy)

ส่วนลดนักศึกษา (10%)

วันวางจำหน่าย (Release Date)

09/08/2026

บันทึก

ยกเลิก

หน้าจอยืนยันลบ + ผลลัพธ์หลังลบ (Delete)

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

ยืนยันการลบ

!

คุณแน่ใจหรือไม่?

การดำเนินการนี้จะลบข้อมูลเกมออกจากระบบอย่างถาวร

ชื่อเกม

Elden Ring (673380052-5 SEC 2)

แนวเกม

Action RPG

แพลตฟอร์ม

PC / PS5

คะแนน

★ 9.9

ราคา

1790.00 บาท

ยืนยันลบ

← ยกเลิก

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

+

เพิ่มเกมใหม่

✓

ลบเกมสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Elden Ring (673380052-5 SEC 2)	Action RPG	PC / PS5	★ 9.7	09/08/2026	ส่วนลดนักศึกษา (10%)	1611.00 1790.00	

ทั้งหมด 1 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

หน้าจอ Database (pgAdmin หรือ terminal psql)

```
lab7demo=# SELECT * FROM games;
 id | discount_type | genre | platform | price | rating | release_date | title
-----+-----+-----+-----+-----+-----+-----+-----
  7 | STUDENT      | Action RPG | PC / PS5 | 1790 | 9.7 | 2026-08-09 | Elden Ring (673380052-5 SEC 2)
(1 row)
```

lab7demo=#

Select Postgres Server

Not Committed Yet

Ln 23, Col 26

Spaces: 4

UTF-8

LF

{ }

Java

Sign In

Go Live